

■ Design a Distributed Job Scheduler

System Design Interview | Asynchronous Systems

algomaster.io | Deep Dive Notes

■ Key Requirements

Functional	Schedule one-time and recurring (cron) jobs; Execute at specified time; At-least-once execution; Job status tracking; Retry on failure; Cancel/pause jobs
Non-Functional	Handle millions of jobs; Trigger within 1s of scheduled time; No missed executions; No duplicate execution (exactly-once preferred); Fault tolerant
Scale	Airflow, Quartz, AWS EventBridge; jobs range from milliseconds to hours; distributed workers for parallel execution

■ High-Level Architecture

- Job Store (DB): persists all job definitions, schedule, status, next_run_time — PostgreSQL with index on next_run_time
- Scheduler Service: polls DB for due jobs ($\text{next_run_time} \leq \text{now}$); claims jobs atomically; enqueues to Worker Queue (Kafka/SQS)
- Worker Pool: stateless workers consume from queue; execute job logic; update status back to DB on complete/fail
- Coordinator: Zookeeper/etcd-based leader election; only one scheduler node polls at a time to avoid duplicate scheduling

■ Cron & Time-Based Scheduling

- **Cron Parsing:** Parse cron expression (0 9 * * MON) to compute next_run_time; store next_run_time in DB; after execution compute and update next occurrence
- **Polling Loop:** Scheduler polls: `SELECT * FROM jobs WHERE next_run_time <= NOW() AND status='SCHEDULED' LIMIT 100 FOR UPDATE SKIP LOCKED` — atomic claim
- **SKIP LOCKED:** PostgreSQL feature: skip rows locked by other transactions; enables multiple scheduler nodes without duplicate picks
- **Clock Skew:** Use DB server time (`NOW()`) not local time to avoid clock drift across scheduler nodes causing missed or duplicate triggers
- **Delay Tolerance:** Build in 1-5s tolerance: jobs due within window are triggered; alert if scheduler lag > 10s

■ Exactly-Once Execution

- Claim step: `UPDATE jobs SET status='RUNNING', worker_id=X, started_at=NOW() WHERE id=Y AND status='SCHEDULED'` — atomic ownership
- Heartbeat: worker updates heartbeat_at every 30s; scheduler detects dead workers (no heartbeat > $2 \times$ interval) and re-queues job
- Idempotent jobs: design job logic to be safe on re-run (check-then-act, use job_execution_id as idempotency key)

- Execution log: separate job_executions table (job_id, execution_id, worker_id, started_at, ended_at, status, result) — full audit trail

■ Retry & Failure Handling

- Configurable retry policy per job: max_retries, backoff strategy (fixed/exponential/jitter), retry_on_exception types
- After max retries: mark job as FAILED; publish to DLQ; alert ops; optionally send notification to job owner
- Timeout: if job runs longer than max_duration → worker killed; job marked TIMEOUT and retried or failed
- Dependent jobs (DAG): job B starts only after job A succeeds; store dependency graph; check parent status before enqueue

■ Scheduling Approaches

DB Polling	Priority Queue (Redis ZSET)
✓ Simple and reliable	✓ Sub-second precision
✓ Works with any DB	✓ ZPOPMIN is $O(\log N)$
✓ Poll interval = min latency	✓ In-memory (fast)
✓ Scales to millions of jobs	✓ Need persistence layer too

■ Observability & Operations

- Metrics: jobs scheduled/min, jobs succeeded/failed, execution latency p50/p99, scheduler lag, worker queue depth
- Dashboard: next scheduled jobs, currently running jobs, recently failed jobs, DLQ size
- Alerting: scheduler lag > 30s, failure rate spike, DLQ growth, worker pool exhaustion
- Admin UI: manually trigger job, pause/resume schedule, view execution history and logs, adjust cron expression